

Overall Course Goal

Build software

Build software

With other people

Build software

With other people

Effectively

Build software
With other people
Effectively
Over time

OK SOME EXTRA
TO BUILD, BUT
WE'LL BE ABLE
REUSE IT FOR ALL OUR
FUTURE PROJECTS.



TO ENSURE YOUR
CODE IS NEVER REUSED

LET'S NOT OVERTHINK
IT. IF THIS CODE IS STILL
USEFUL *THAT* FAR IN THE
FUTURE, WE'LL HAVE
BIGGER PROBLEMS.



HOW TO ENSURE
YOUR CODE LIVES FOREVER

Course Philosophy

Learn the Theory (Version Control, Dependency Management, Design Patterns, etc)

Build a useful foundation

Practice with Specific Tools (Git, Gradle, Java, etc)

Current industry standard versions because why not

Development Ecosystem

Version Control + Local Checkout + Build System Management + IDE = Success

Specifics:

Github + Github Desktop + Gradle + Eclipse

Supported vs. Allowed

Common distinction in industry

Allowed vs Not Allowed: Security, Dogfooding

Supported vs Not Supported: Standard workflow, existing expertise

Are Those Exact Specifics Required?

Github = Required (others not allowed)

Github Desktop = recommended, but any Git client will work

Supported: Github Desktop, command-line git

Can use multiple clients at the same time, low risk to experiment here

Gradle = Required

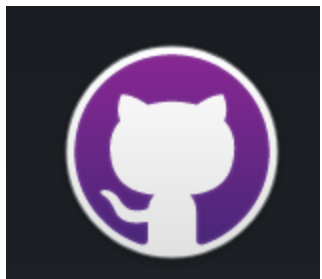
Maven: technically allowed, but a lot more work, **unsupported**

Eclipse = __strongly __ recommended

VSCode: not allowed, because it's missing required features

IntelliJ: allowed, __unsupported __ (FYI, they embrace 'move fast and break things')

Basic Workflow



Demo! (Time Permitting)

Grab the starter code

Import into IDE

Commit changes

Invite users to a repo

Code Reviews: Cheap Way to Avoid Bugs



Structuring Code For Code Review

Style **conventions** are aspects of code not enforced by the compiler, but which make code much more approachable:

Ex:

```
int getWidth() {  
    return width;  
}
```

and

```
rectangle.getWidth()
```

vs

```
int Width(){return a;}
```

Structuring Commits For Code Review

Manageable size (less than 200 lines of non-auto-generated code)

Detailed description, descriptive summary message

Single feature/bug/task

	COMMENT	DATE
○	CREATED MAIN LOOP & TIMING CONTROL	14 HOURS AGO
○	ENABLED CONFIG FILE PARSING	9 HOURS AGO
○	MISC BUGFIXES	5 HOURS AGO
○	CODE ADDITIONS/EDITS	4 HOURS AGO
○	MORE CODE	4 HOURS AGO
○	HERE HAVE CODE	4 HOURS AGO
○	AAAAAAA	3 HOURS AGO
○	ADKFJSLKDFJSDKLFJ	3 HOURS AGO
○	MY HANDS ARE TYPING WORDS	2 HOURS AGO

Giving a Code Review

- Step 1: Grok the code
- Read the commit summary & description
- Skim through the whole change
- Do a second pass for deeper understanding
 - Make notes of anything that looks unclear
 - If necessary, sketch out a diagram/outline of what the change is doing

Step 2: Comment on larger issues

Be nice! Criticism always feels harsher on the receiving end than the giving end

A code review is a **conversation** - for larger changes, expect to have multiple rounds of commits/reviews

Topics:

Demo!

Leave inline comments

Resolve comments



Using a Linter

Code reviews that spend a lot of time on nits are sub-optimal

Fortunately, some formatting issues are easy to spot programmatically

A **linter** is something that cleans up code "lint" - the small bits of easily removed fluff to leave clean and polished code behind

checkstyle is an open source, easily configured version, which we'll be using.

Using Checkstyle With Github

We're going to hook up a custom checkstyle config to the status checks in Github

Every Pull Request will automatically trigger these checks

Don't request a review/don't leave a review until these checks pass

They should be very quick

This is an example of Continuous Integration (CI)

Note : there are lots of ways to call checkstyle, including from gradle and as an Eclipse plugin. Feel free to add additional calls/checks for your project!

Continuous Integration

Shorten the loop between code being written and being deployed to production

Automates error detection, shipping, and rollbacks

Complexity ranges from very simple (checkstyle checks) to very complex (automated canary rollouts with rollbacks if certain metrics spike)

Benefits:

Faster feedback

Easier to launch features

Higher quality code



Pair Programming

Combines the coding and code review steps

Requires 2 people looking at the same screen with low-friction instant communication (ideally in-person, but screenshare with discord also works; email does **not** count as low-friction instant communication, though)

One person types, the other reviews

Good for the first several commits of a large, complex architecture

avoids latency and context switching from code reviews

When committing, note in the **commit description** that the code was pair-programmed (and with who); this should line up with the person who then __approves__ the pull request

Where/How to Write Code

Ex: Writing a 20 page research paper

You **could** use a plain text editor, but you probably wouldn't

Instead, google docs/libre office/ms word:

Spell check

Grammar check

Formatting

Text suggestions

These tools are **integrated** into your word processing **environment**

Same for code! Use an **I**ntegrated **D**evelopment **E**nvironment (IDE)

Constant compilation/compile error readout

IDE for Java



Recommended: Eclipse

Fully featured

Easy to migrate from this to other tools

Optional: IntelliJ

Industry standard

Not so great if you don't have a build tools team

Do Not Use: Visual Studio Code (VSCode)

Unlock the Power of the IDE

Emacs/notepad is fine for C code, but modern languages are designed to be written with an IDE

Solves the complexity to type vs complexity to read tradeoff

Minimizes "toil"

Allows for easy navigation of object-oriented code

Shortcuts in the IDE are HUGE for productivity, they are absolutely worth learning

Deterministic vs Stochastic Code Generation

Deterministic: Compiler, Many IDE tools

No need to double-check output ("Just glance through the .class bytecode to make sure it's right" said no one ever)

Solves specific problems

Non-deterministic (stochastic): LLMs, other ML models, stack overflow

Must double-check output

Can attempt any problem, but no guarantee of successfully solving

⇒ For any problem that can be solved deterministically, **prefer that option**

This requires **knowing when those options exist**

Organize Imports

Eclipse shortcut : ctrl + shift + o

Doesn't just "organize" existing imports

Searches the classpath for any unreferenced types, and attempts to add them

Classic API design:

Import package structure is an **implementation detail**

The library classes/interfaces are the **API**

Organize imports = "Hey IDE, I want this class. Go find it for me"

Auto-Refactor

Eclipse shortcut :

Rename: alt + shift + r

Extract method: alt + shift + m

Rename Classes/parameters/methods/interfaces

Extract methods

⇒ Reduces the cost of readability/maintainability fixes

Autocomplete

Eclipse shortcut : ctrl + space

Long variables/methods

⇒ Prioritize readability without a hit to productivity

Forgot a method? The IDE is here to help!

Eclipse shortcut : type . and wait

⇒ Efficiently use a large number of libraries

Navigate Object Hierarchy

Eclipse shortcut : ctrl + click

Object Oriented Programming has tons of classes, interfaces, and instance variables that all interact across files

Navigating in the file structure is sloooooow

⇒ context switching is expensive

⇒ reasoning about the whole system is difficult

Better: click-through navigation in the IDE

And Many More!

Deterministic generation (no generative AI, please!):

hashCode/equals/toString

getters/setters

try/catch blocks

unimplemented methods

Auto-format files

"On save" actions

Etc.

Let's Review!

What you should know from this week that might be on an exam:

Version Control : What is it, what are branches, when would you use a feature vs. release branch

Github Workflow : What is a Pull Request, what is branch protection, what are status checks

Code Reviews : Why have them, the 3 features of a good code review process, tips for structuring code and commits for review, and the 3 steps in giving a code review

Style Conventions : What are some common ones, why are they important

Continuous Integration : What are some examples, what are some benefits

Pair Programming : What is it, and what problem with standard code reviews